

INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

ESCOM



Materia: Bases de Datos

Profesor: Gabriel Hurtado Aviles

Grupo: 3CV4

Título:

Práctica 1: "Modelo Entidad Relación"

Miembros de Equipo:

Jiménez Sánchez Taylin

Introducción

En este documento se realizara algunos ejercicios con el objetivo de familiarizarse para realizar las practicas con mayor control.

Práctica 1: "Modelo Entidad Relación"

Objetivo: Que usted describa los sistemas de base de datos a partir de sus aspectos básicos, construya el entorno de trabajo que empleará durante todo el semestre —control de versiones y un sistema gestor de bases de datos ejecutado en un contenedor—, sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda la evidencia se entrega en un repositorio de Git.

Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustentan la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado cómo trabajó usted, no solo el resultado final.

Ejercicio 2. El sistema gestor en un contenedor: Docker

Parte A. Investigación. En media cuartilla explique:
1. Qué es un contenedor y en qué se diferencia de una máquina virtual, en términos de arranque, tamaño y aislamiento.
2. Defina: imagen, contenedor, volumen y puerto publicado.
3. Por qué el volumen es indispensable y qué ocurre exactamente si no lo declara.

Un contenedor es una unidad ligera que reúne una aplicación con todos los elementos y dependencias que necesita para funcionar, utilizando el mismo sistema operativo del equipo anfitrión.

Las principales diferencias entre un contenedor y una máquina virtual (VM) son las siguientes:

- **Arranque:** Los contenedores pueden iniciarse en pocos segundos, ya que comparten el kernel del sistema operativo del host. En cambio, una máquina virtual debe iniciar su propio sistema operativo.
- **Tamaño:** Los contenedores ocupan menos espacio porque no necesitan incluir un sistema operativo completo, a diferencia de las máquinas virtuales.
- **Aislamiento:** Las máquinas virtuales proporcionan un aislamiento a nivel de hardware virtualizado, mientras que los contenedores aíslan los procesos y recursos, pero utilizan el mismo kernel del sistema operativo.

La **imagen** es una plantilla que contiene los elementos necesarios para crear un contenedor.
Un **contenedor** es una instancia en ejecución de esa imagen.
Un **volumen** es un mecanismo que permite almacenar información de manera persistente, independientemente del ciclo de vida del contenedor.
Un **puerto publicado** permite conectar un puerto interno del contenedor con uno del host para que se pueda acceder a la aplicación desde el exterior.

Parte B. Práctica. Levante un sistema gestor de bases de datos sin instalarlo en su sistema operativo:

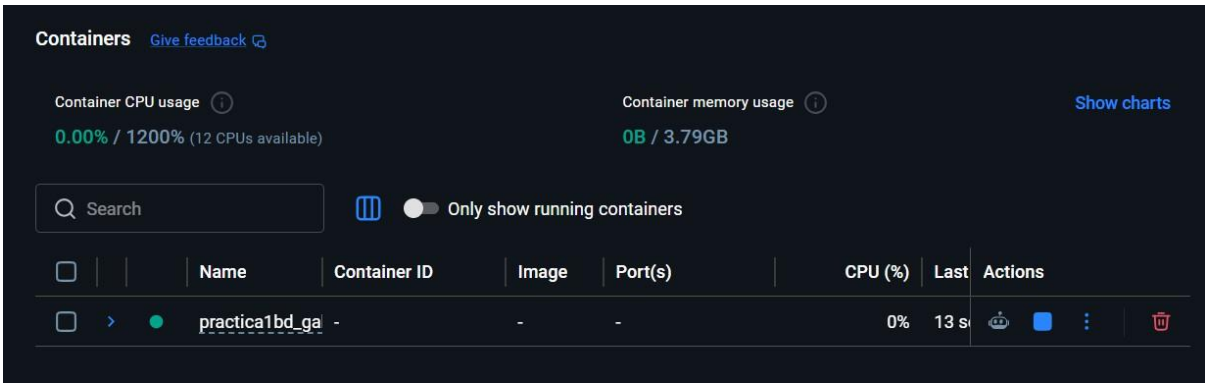
1. Instale Docker Desktop, o bien Podman si prefiere una alternativa sin demonio privilegiado.
2. Levante el gestor de su elección en un contenedor. Se recomienda PostgreSQL por ser el que se usará en el curso.
3. Declare la configuración en un archivo compose.yaml y versiónelo en el repositorio. No basta con haber escrito el comando una vez en la terminal.
4. Conéctese con un cliente (psql, DBeaver o pgAdmin) y cree una base de datos vacía llamada practica1.
5. Demuestre la persistencia: detenga y elimine el contenedor, vuelva a levantarlo con el mismo volumen y compruebe que la base de datos sigue existiendo.

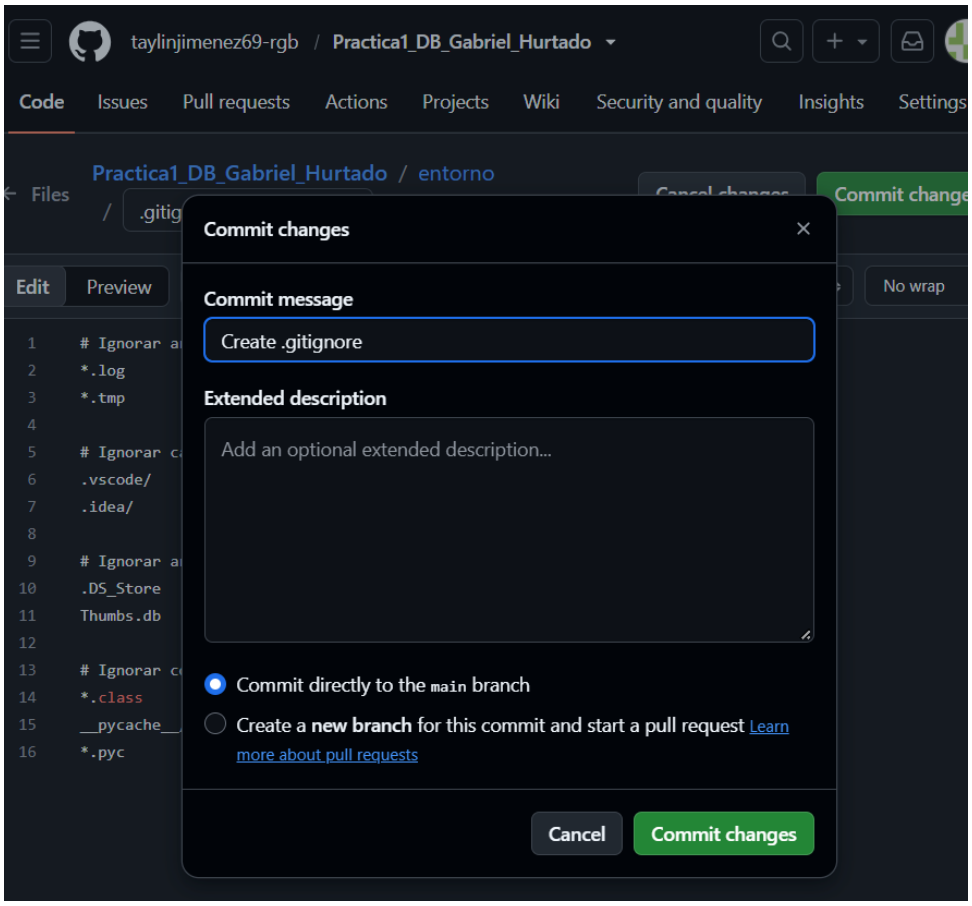
Comando mínimo de referencia:

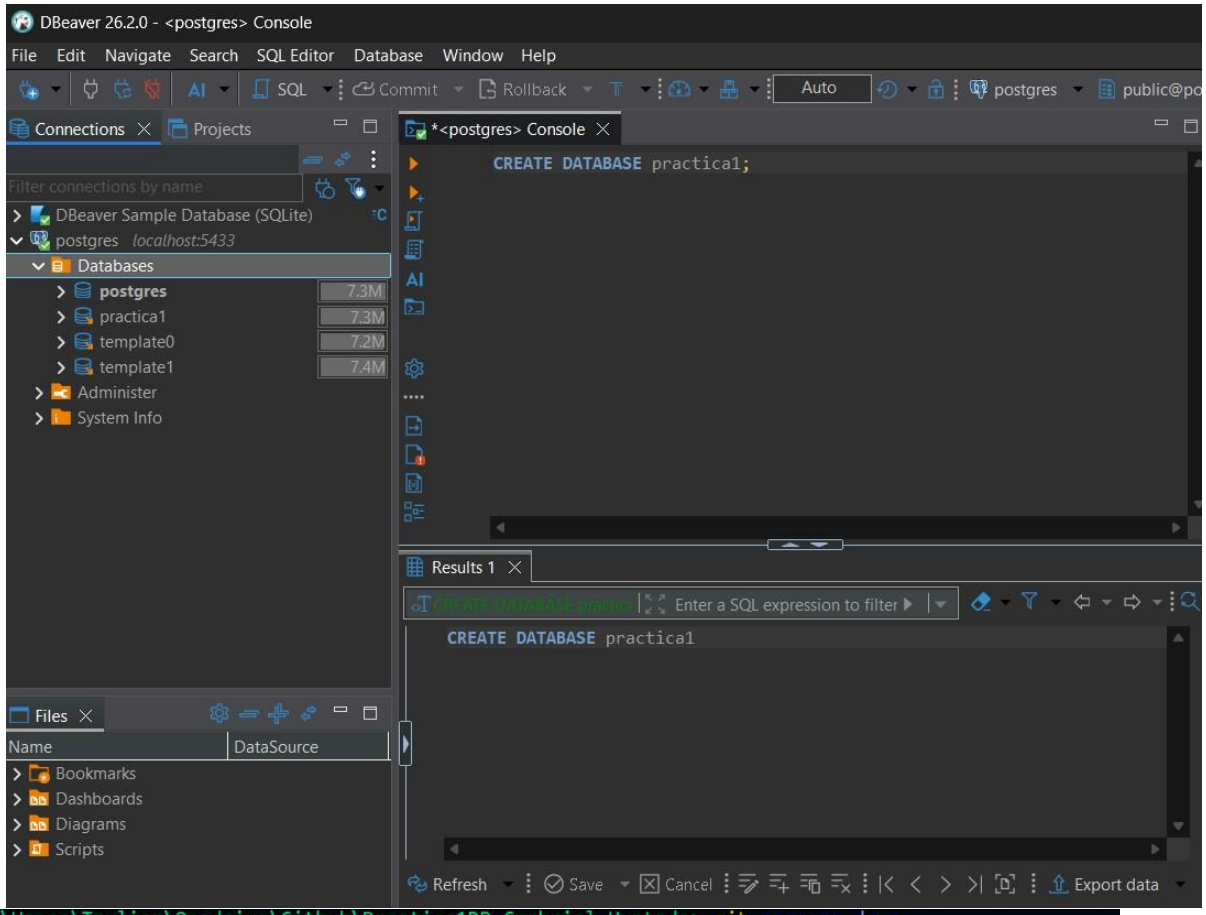
```
docker run -d --name pg-practica1 \
 -e POSTGRES_PASSWORD=practica1 \
 -p 5432:5432 \
 -v pgdata:/var/lib/postgresql/data \
 postgres:16
```

Evidencia: el archivo compose.yaml versionado, una captura de la conexión al gestor y una captura de la prueba de persistencia.

Nota. Si su equipo no admite virtualización, documente por escrito el motivo, realice la instalación nativa del gestor y entregue la investigación de la Parte A completa.

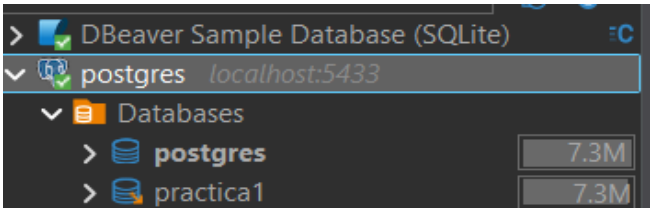

Containers
Give feedback
Container CPU usage
0.00% / 1200% (12 CPUs available)
Container memory usage
0B / 3.79GB
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Last
Actions
practica1bd_ga
-
-
-
0%
13 s


taylinjimenez69-rgb / Practica1_DB_Gabriel_Hurtado
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Practica1_DB_Gabriel_Hurtado / entorno
Files
Commit changes
Edit
Preview
No wrap
Ignorar a
*.log
*.tmp
Ignorar c
.vscode/
.idea/
Ignorar a
.DS_Store
Thumbs.db
Ignorar c
*.class
__pycache__
*.pyc
Commit changes
Commit message
Create .gitignore
Extended description
Add an optional extended description...
Commit directly to the main branch
Create a new branch for this commit and start a pull request Learn more about pull requests
Cancel
Commit changes




```
PS C:\Users\Taylinn\Onedrive\Github\Practica1DB_Grabriel_Hurtado> git process down
[ 1] 23% ]%]
 * Container DB_proces23a                          Removed
 * Network Practica1DB_Grabriel_Hurtado_default    Removed
PS C:\Users\Taylinn\Onedrive\Github\Practica1DB_Grabriel_Hurtado> docker compose down
Get-Process : No se encuentra ningún parámetro que coincida con el nombre del parámetro 'a'.
En línea: 1 Carácter: 4
+ <<<<<
+    ->>
    + CategoryInfo          : InvalidArgument: (:) [Get-Process], ParameterBindingException
    + FullyQualifiedErrorId : NamedParameterNotFound,Microsoft.PowerShell.Commands.GetProcessCommand

PS C:\Users\Taylinn\Onedrive\Github\Practica1DB_Grabriel_Hurtado> docker compose up -d
[+] 1/2
 * Network Practica1DB_Grabriel_Hurtado_default    Created
 * Container DB_proces23a                          Started
PS C:\Users\Taylinn\Onedrive\Github\Practica1DB_Grabriel_Hurtado>
```



Conclusión

Jimenez Sanchez Taylin:

Durante, la realización de esta práctica permitió consolidar los fundamentos prácticos de la contenerización y su impacto directo en la infraestructura moderna de bases de datos. Al implementar herramientas como Docker y Docker Compose, se experimentó de primera mano la ventaja de desplegar entornos aislados y portables, eliminando los problemas de compatibilidad del sistema.

El manejo estratégico de volúmenes demostró ser un componente crítico para solucionar la naturaleza volátil de los contenedores, asegurando la durabilidad e integridad de los datos en gestores como PostgreSQL. Este enfoque técnico favorece la automatización, la reproducibilidad y las mejores prácticas en el desarrollo de software orientado a datos.

Bibliografía

- Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/en/v2
- Docker. (2024). Try Docker Compose. Docker Documentation. https://docs.docker.com/compose/gettingstarted/
- Docker. (2024). What is a container? Docker Documentation. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
- PostgreSQL Global Development Group. (2024). PostgreSQL 16.0 Documentation. PostgreSQL. https://www.postgresql.org/docs/16/index.html
- **Elmasri, R., & Navathe, S. B.** (2015). *Fundamentals of Database Systems* (7.ª ed.). Pearson.
- **Humble, J., & Farley, D.** (2010). *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- **Merkel, D.** (2014). *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014(239), 2.
- **Turnbull, J.** (2014). *The Docker Book: Containerization is the new virtualization*. James Turnbull.